## Algorithm

↳ Sequence of finite steps to perform some specific task.

Example → a , b

Multiplication of two number
1. Take two number → a & b.
2. Take C = a * b.
3. Return C.

## Properties of an Algorithm

1) Terminate after finite amount of time
2) Produce atleast One Outcome.
3) Independent of any programming language
4) Unambiguous / Deterministic
   ↳ Independent of time

Example → while (True):
print ("hi there!") → not an Algorithm

## Steps to construct an Algorithms

1) Problem Defination
2) Design algorithm → Divide & Conquer, Greedy Algo, Dynamic Programmic & Many More
3) Draw Flow Chart
4) Testing
5) Implementation

6) Analysis

Asymphatic Notation

- Big O → Worst Case Scenario
- Omega → Best Case Scenario
- Theta → Average Case Scenario

Analysis
- Apostiary Analysis (Exact result)
  - language of Compiler & type of hardware
- Apriori Analysis
  - Independent
  - Approximate Answers

Apriori Analysis → Order of Magnitude of Statement

Example 1) $(x = y + z)$ → Constant C

→ O(1) (No Loop)

Example 2) $x = y + z$ → 1 time

```
for( int i = 0; i < n; i++){ → n time
    x = y + z;
}
```

$(n+1)$ times ⇒ O(n)

Example - 3) $x = y + z$ → 1 times

```
for( i → 0 to n-1){
    x = y + z; → n times
}
```

```
for ( i → 0 to n-1){
    for ( i → 0 to n-1) {
        x = y+z;   → n² time
    }
}
```

Example - 4)

```
n = 5;
i = n;
while ( i > 1){ → (n-1) time ⇒ O(n)
    i = i-1;
    Cout << " Hi There! "
}
```

Example 5)

```
i = n
while ( i > 1){  ⟶ n/3 times ⇒ O(n)
    i = i - 3;
    cout << " hi There! ";
}
```

Example 6)

```
i = 1;
while( i < n){
    i = 2 * i  ⟷  i = 3 * i
    cout << i ;
}
```

$O(\log_2 n)$ ⟵ ⟶ $O(\log_3 n)$

$O(\log n)$

Example 7) $i = n$

while $(i > 2)$ {

$i = i^{1/2}$ ⟶ $[n^{1/2^K} = 2]$ stopping criteria

↓

taking log both side

$\log n^{1/2^K} = \log_2^2 1$

$\frac{1}{2^K} \log n = 1 \Rightarrow \log n = 2^K$ (taking log)

$\Rightarrow \log_2(\log_2 n) = \log_2^2 2^K$

$\Rightarrow \boxed{K = \log_2(\log_2 n)}$

1) $i = n$

while $(i > 2)$ { ⟶ $n^{1/25^K} = 2$

$i = i^{1/25}$ $\Rightarrow \log_2 n^{1/25^K} = \log_2^2 1 \Rightarrow \frac{1}{25^K} \log n = 1$

}

$\Rightarrow \log n = 25^K \Rightarrow \log_{25} \log_2 n = \log_{25}^{25^K}$

$\Rightarrow \boxed{K = \log_{25} \log_2 n}$

2) $i = 29$

while $i < n$: ⟶ Step by Step Analysis

$i = 29 \rightarrow 29^{23} \rightarrow 29^{23^2} \rightarrow 29^{23^3} \ldots\ldots 29^{23^K}$

$i = i^{23}$

↳ hence $[29^{23^K} = n]$

$\rightarrow \log_{29} 29^{23^k} = \log_{29} n \Rightarrow 23^k = \log_{29} n$

$\rightarrow \log_{23} 23^k = \log_{23} \log_{29} n \Rightarrow K = \log_{23} \log_{29} n$ ✓

$\rightarrow O(\log\log n)$

3) $i = 1$

while $i < n$:

$i = 2 * i$
$i = 3 * i$ $\rightarrow i = 6 * i \rightarrow 6^k = n$

$\Rightarrow [k = \log_6 n] \rightarrow O(\log n)$ ✓

Complexity Classes:

1) Constant - $O(1)$
2) Logarithm - $O(\log n)$
3) Linear $\rightarrow O(n)$
4) Quadratic $\rightarrow O(n^2)$
5) cubic $\rightarrow O(n^3)$
6) polynomial $\rightarrow O(n^c)$
7) Exponential $\rightarrow O(c^n)$ - $O(2^n)$
8) $n! < n^n$
   $2^n < n^n$
   $n! > 2^n$

$2^n < n! < n^n$

9) $\log n < n$

$(\log n)^2 < n$

$(\log n)^3 < n$

$(\log n)^{1000} < n$

$\boxed{(\log n)^{\log n} > n}$ ⟶ True

10) $2^n < 3^n$

$2^n < (2 \times 1.5)^n$

11) $\log_2 n > \log_3 n$

Recurrence Relation

- Substitution
- Recursive
- Master Theory

Substitution Method

1) $$T(n) = \begin{cases} 1 & , n = 1 \\ T(n-1) + n & , n > 1 \end{cases}$$

↳ A function calling itself

$\Rightarrow \boxed{T(n) = T(n-1) + n}$ —— 1st time

↳ Substitute

$T(n-1) = T(n-1-1) + n - 1$

$T(n-1) = T(n-2) + n - 1$

$$T(n) = T(n-2) + n-1 + n \rightarrow 2^{nd} \text{ time}$$

$T(n-1)$

$$T(n) = T(n-3) + n-2 + n-1 + n \rightarrow 3^{rd} \text{ time}$$

$T(n-2)$

↓ K times

$$T(n) = T(n-K) + n-K-1 + n-K-2 \ldots + n-1 + n$$

so for stopping criteria $[n-k=1]$ base case

hence $n-1 = k$ replacing...

$$T(n) = T(n-(n-1)) + n-(n-1) + 1 + n-(n-1)+2 + \ldots + n-1 + n$$

$$\Rightarrow T(1) + 2 + 3 + \ldots + n-1 + n$$

$$\Rightarrow 1 + 2 + 3 + \ldots + (n-1) + n$$

↳ Sum of n natural numbers

$$\Rightarrow \frac{n(n+1)}{n} = \frac{n^2+n}{2} = O(n^2)$$ ∴

Example - 2) $T(n) = \begin{cases} 1 & , n = 1 \\ T(n-1) + \frac{1}{n} & n > 1 \end{cases}$

$$T(n) = T(n-1) + \frac{1}{n} \rightarrow 1^{st} \text{ time}$$

$$T(n) = T(n-2) + \frac{1}{n-1} + \frac{1}{n} \rightarrow 2^{nd} \text{ time}$$

$$T(n) = T(n-3) + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n} \rightarrow 3^{rd} \text{ time}$$

$\downarrow$ k times

$$T(n) = T(n-k) + \frac{1}{n-k+1} + \frac{1}{n-k+2} + \cdots + \frac{1}{n-1} + \frac{1}{n}$$

$n - k = 1 \Rightarrow \boxed{n - 1 = k}$

$$\boxed{T(n) = T(1) + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots + \frac{1}{n-1} + \frac{1}{n}}$$

$$= 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1} + \frac{1}{n}$$

$$= \Theta(\log(n))$$

Example - 3)

$$T(n) = \begin{cases} 1 & n = 1 \\ n * T(n-1) & , n > 1 \end{cases}$$

↳ fibo recurrence relation

$$T(n) = T(n-1) * n$$

$$T(n) = T(n-2) * n-1 * n$$

$$T(n) = T(n-3) * n-2 * n-1 * n$$

↓ K times

$n - k = 1 \Rightarrow k = n - 1$

$$T(n) = T(n-K) * n-k+1 * n-k+2 * \ldots * n-1 * n$$

$$= T(n-(n-1)) * n-(n-1)+1 * n-(n-1)+2 * \ldots * n-1 * n$$

$$= T(1) * 2 * 3 * \ldots * n-1 * n$$

$$\Rightarrow 1 * 2 * 3 * \ldots * n-1 * n$$

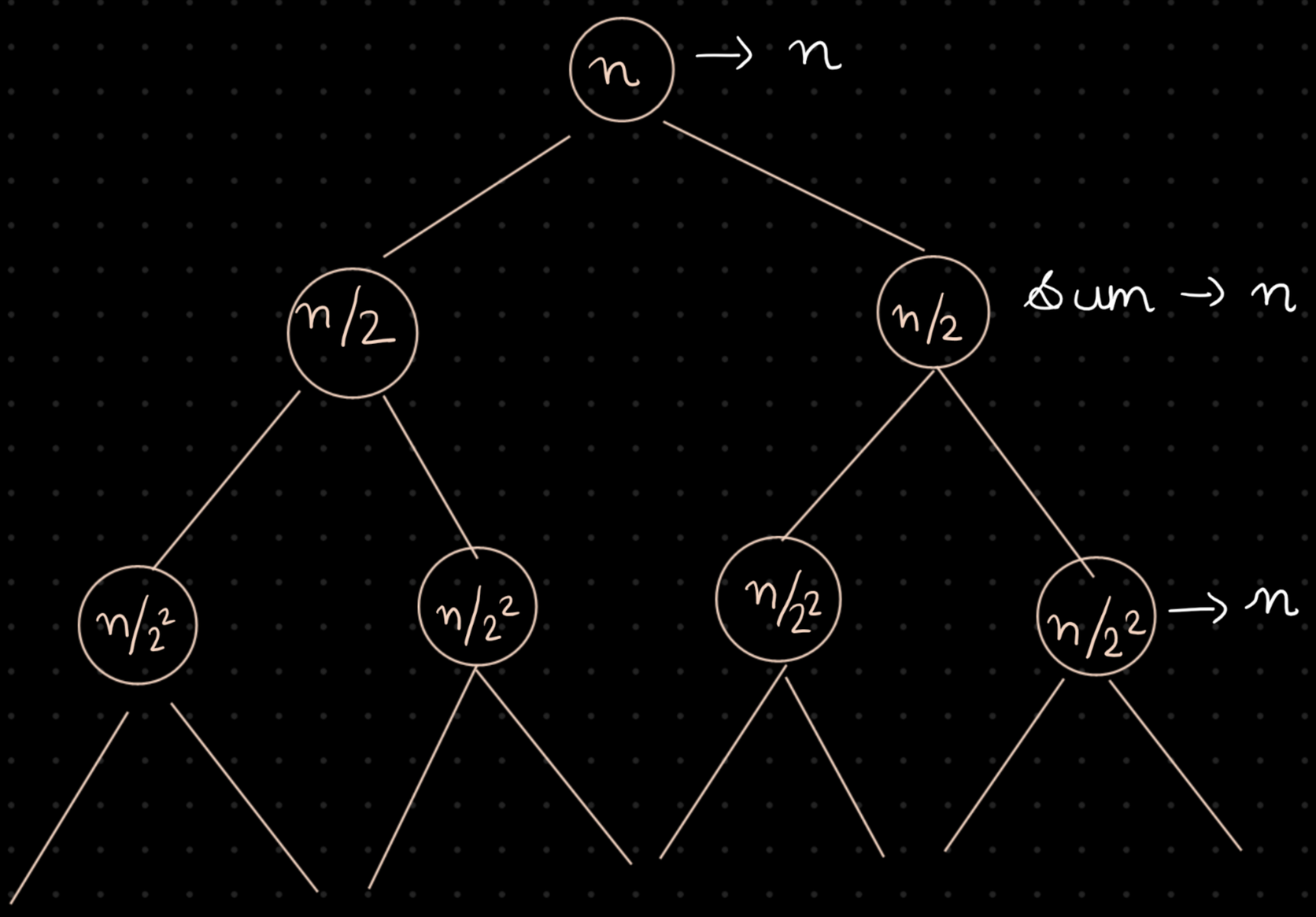
$$\Rightarrow O(n!) \Rightarrow O(n^n)$$

# Recursive Tree Approch

↳ More than one recursive term in the recursive relation.

Example $T(n) = \begin{cases} 1 \\ T(n/2) + T(n/2) + n \end{cases}$

$$T(\underline{n}) = T(n/2) + T(n/2) + \underline{n}$$




K times

Left side

$$\frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

Right side

$$\frac{n}{2^k} = 1$$

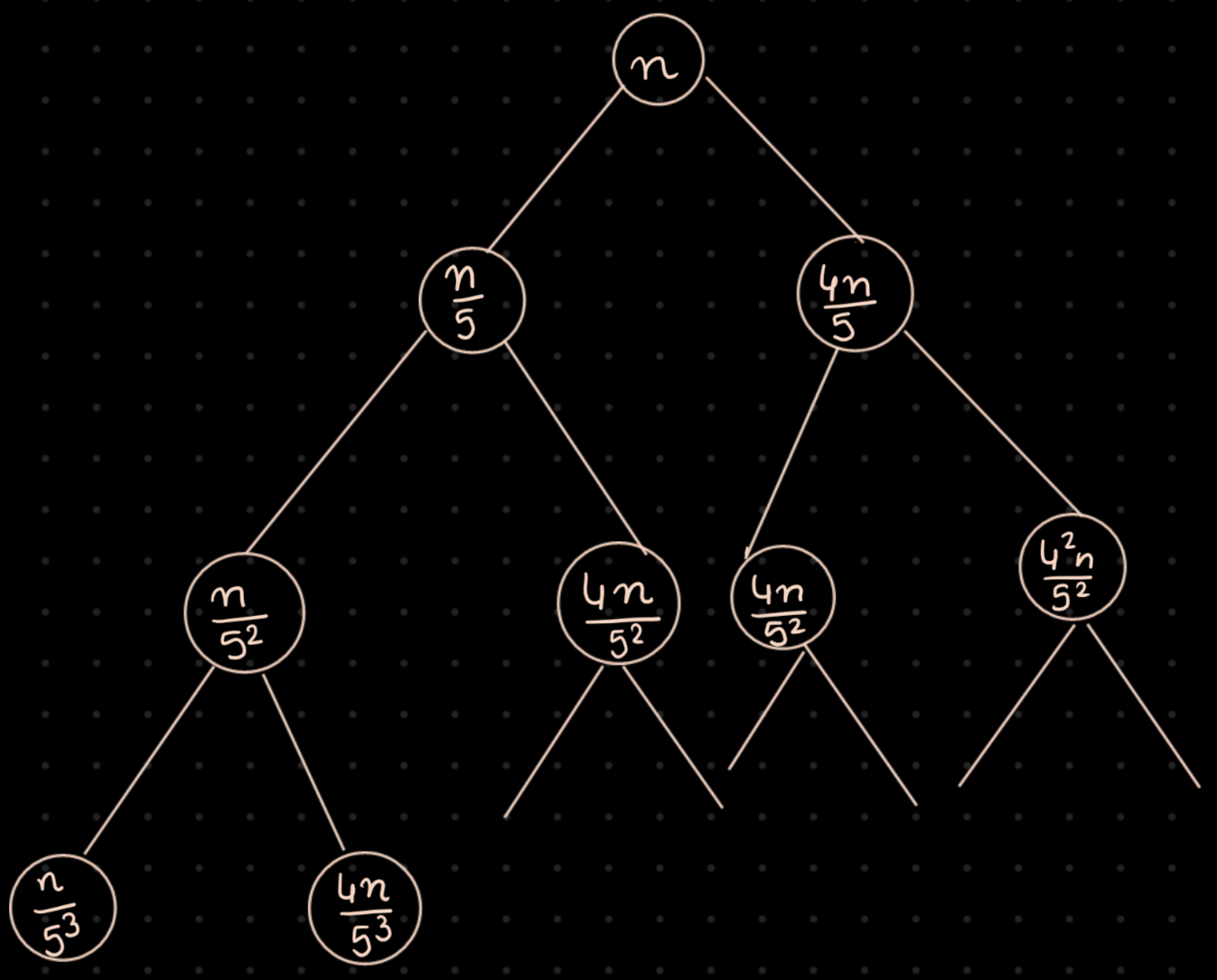
$$n = 2^k$$

$\Rightarrow \log_2 n = \log_2 2^K$ $\qquad\qquad$ $\Rightarrow \log_2 n = \log_2 2^K$

Ans → $O(n * K) \Rightarrow O(n \log_2 n)$ A

Example -2)

$$T(n) = \begin{cases} 1 & , n = 1 \\ T(n/5) + T\left(\frac{4n}{5}\right) + n & , n > 1 \end{cases}$$




Left Side

$$\frac{n}{5^{K_L}} = 1$$

$\Rightarrow n = 5^{K_L}$

$\Rightarrow \boxed{\log_5 n = K_L}$

Right Side

$$\frac{n}{\left(\frac{5}{4}\right)^{K_R}} = 1$$

$\boxed{K = \log_{5/4} n}$ → greater

Time Complexity → $O(n * K_R)$

$\Rightarrow O(n \log_{5/4} n)$ A

# Basic Properties of Logarithm

1) $a^x = n \Rightarrow \log_a a^x = \log_a n$

$x = \log_a n$

2) $\log_b 1 = 0$

3) $\log_a a = 1$

4) $\log_a (p/q) = \log_a p - \log_b q$

5) $\log_a pq = \log_a p + \log_a q$

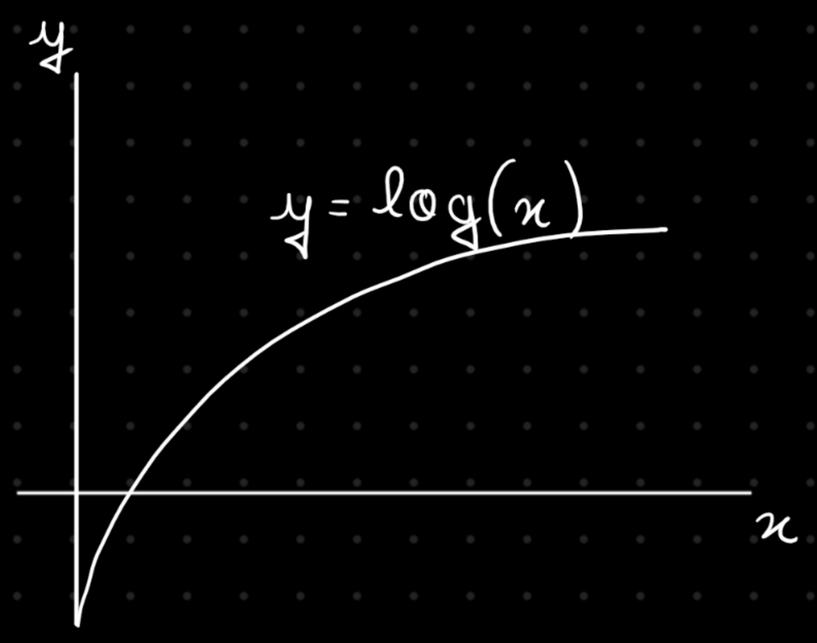
6) $\log_a p^n = n \log_a p$

7) $a^{(\log_a p)} = p \; ; \Rightarrow p^{\log_a 1} = p$

8) $\log_a^n = \log_b^n * \log_a^b$

9) $\log_a^b = \log_n^b / \log_n^a$

10) Lower the base in logarithmic
$\longrightarrow$ higher the value



# Master Theorem

$$T(n) = aT\left(n/b\right) + f(n)$$

$$f(n) = O\left(n^k \log^p n\right)$$

$$T(n) = 2T\left(n/2\right) + n^2$$

$a = 2$

$b = 2$

$K = 2$

$p = 0$

case - 1) $\log_b^a > k$

# Data Structure

Array ⟶ Algorithms

↓ ↓

Data Structure | Divide and Conquer

↳ Recursion

1) Linear
2) Non Linear

collection of items in a contiguous manner

Base Address - 1000

| 85 | 75 | 70 | 65 | 98 | 99 | 50 | 33 | 47 | 77 | 42 | 71 | 69 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| ↓ | 1004 | 1008 | - | - | - | - | - | - | - | - | - | - |

Index

size(int) = 4 bytes

Address of $12^{th}$ index = Base Address + (i - lower bound) * size of each element

$\Rightarrow \boxed{1000 + (9-0) * 4}$

$\Rightarrow$ 1036 Ans

## Searching

[ 20 45 27 47 55 67 75 88 90 ]

```
for ( int i = 0 ; i < n; i++){
    if ( arr(i) == x) {
        return i
```

}

return -1

$T(n) = O(n)$

↳ Worst Case (Element present near to last Index)

Best case → $O(1)$

↳ Element present near to the first Index

Average case → $\frac{1}{2} \cdot O(n) + \frac{1}{2} \cdot O(1) = O(n)$

## Binary Search

↳ Sorted Array (Ascending or Descending)

[20, 30, 40, 50, 60, 70, 80, 90]

left = 0

right = 7

target = 30

```
BinarySearch( vector<int> &arr , left , right){
while( left <= right ){
    int  mid =  left + ( right - left )/2 ;
    if ( arr[mid] == target ){
        return mid ;
    }
    else if( arr[mid] < x ){
        left = mid + 1 ;
    }
    else {
        right = mid - 1;
    }
  }
  return - 1;
}
```

# Recursion

↳ Calling the same function again inside the method definition

```
#include <iostream>
using namespace std;

int binarySearchRecursive(int arr[], int low, int high, int target){
if( low > high) {
    return -1;
}

int mid = low + (high - low) /2;
if ( arr[mid] == = target ){
      return mid;
} else if ( arr[mid] > target ){
    return binarySearchRecursive ( arr, low, mid
                                   -1, target );
} else {
    return binarySearchRecursive (arr,
```